Report

Prajwal and Vishesh

April 29, 2026

Contents

1 Design and Architecture **3**

2 External Libraries Used **3**

3 Installation and setup **4**

4 Directory structure **4**

5 Launching the game(Usage) **5**

6 Features **6**

7 Code Logic **7**
- 7.1 Generic Class 7
- 7.2 Tic Tac Toe 7
- 7.3 Connect Four 8
- 7.4 Othello 8
- 7.5 Authentication System 9
- 7.6 Leaderboard 9

8 Retrospective **10**
- 8.1 axis 10
- 8.2 Othello's logic 10
- 8.3 diagonal check win 10
- 8.4 Text alignment with varying length of player names 10

8.5 Arrangement according to the background image for various games . . . 11

9 What would we do with 10 more hours **11**

9.1 Add new games . . . 11

9.2 Make better UI . . . 11

9.3 Optimise . . . 11

9.4 Add settings . . . 11

10 Images **13**

1 Design and Architecture

The design of the project is pretty simple, straightforward and a block based approach.
The authentication into the game is handled by `main.sh` which takes usernames and passwords and manage them in users.tsv file.
After authentication, the `game.py` is called which has the main code for the games, it has a generic class which is inherited by the three games- tic tac toe, connect four and othello.
The game.py has funcions common in all games such as check win, change turn, current turn player,update history, etc.
It also has some extra functions such as pregame countdown animation, postgame graph update, etc. which are intended for enhancing the UI and design of the game hub. These could have been added in the individual game files, but were added in the general class for code reusability. Hence most of the code possible is tried to be kept in the main file itself.
Other function specific to games are written in the respective game classes, the game files also import the general class.

2 External Libraries Used

- pygame -ce
- numpy
- matplotlib
- sys
- os
- pathlib
- time
- datetime
- subprocess

3 Installation and setup

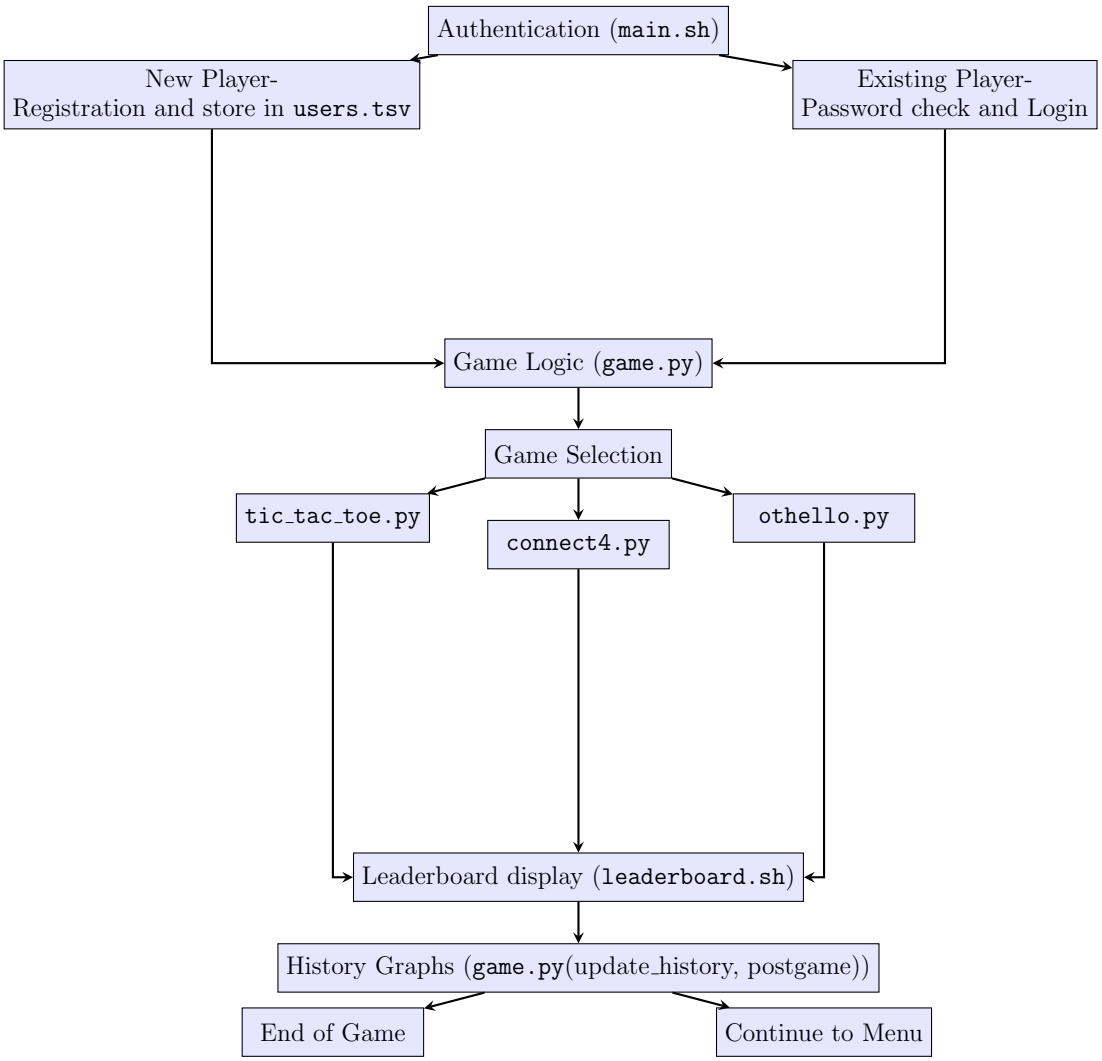
installed python and all the libraries mentioned in "libraries used section".
setup on github (made an account on github website).
code editors used generally - vs code, overleaf.

4 Directory structure

```
hub
├── Fonts and Audio
├── Games
│   ├── pycache
│   ├── connect4.py
│   ├── othello.py
│   └── tictactoe.py
├── Images
├── Report
│   ├── Makefile
│   ├── Report.pdf
│   ├── Report.tex
│   └── references.bib
├── firework frames
├── game.py
├── history.csv
├── leaderboard.sh
├── main.sh
└── users.tsv
```

5 Launching the game(Usage)

After getting all the libraries, you can run the game by running `main.sh` in terminal, it will ask for login or registration of two players, then it will ask to select the game and then it will run the python script.
Basic process flow diagram of the game is as follows-



6 Features

There are various Features we added to make the gameplay more interesting and engaging, such as-

- User authentication system using SHA-256 hashing and tsv file storage
- 2 player games
- Shows history of the players wins and losses
- Uses graphs, etc to show data
- Display of characters in menu with thier player names written below them
- Background music in the game
- Falling animation of the circle in Connect Four
- Sound effects for various actions such as clicking, etc
- Background images which makes it look more attractive
- Extra animation before start of each game
- Added fireworks animation at the end of the game screen -
  It is a loop of frames which initiate randomly and show the animation. Based on random generation and frame shifting.

7 Code Logic

7.1 Generic Class

Here, the code written is the parent code which is inherited by the three games, it has some functions and variables defined,such as-

Variables-
player1 and player2 stored in players array, current turn, board

Functions-
check win, current turn player, change turn

All these are working as their name suggests, easy to understand

There are also functions about Updating History, Postgame update in the graphs using history.csv as data and pregame countdown screen animations using loops and loading multiple images.
Generally all games have similar logic- run in loop, each time taking events and executing, updating the board array, for this calling some general functions like update board, change turn, current turn player, etc. The `draw_board` in each game class prints the board image on which the game will be played, this is done by using the base class's current turn function. There is also a back to menu button which if selected quit it will exited the game class loop and return 0 to `game.py`. Which in turn simply continues its main loop and shows back the menu with no update history or postgame or anything. There is a turn indicating text which updates on each turn. And if the game is completed, the winner (or None in case of draw) is returned to the game.py
Now , for each game, only unique logics are explained below

7.2 Tic Tac Toe

It has a board of size 10*10. the winning criteria is to make a consecutive straight line of atleast 5 same pieces(X or O).
for check win, we first check the rows, then colomns, then diagonal and anti diagonals.
for this, especially diagonals i made a set of all 5*5 submatrices in the board using `nplibstride_tricksgliding_window_view`, then checked these condi-

tions for each matrix.
some general functions used are- draw line/circle, update board, etc which track the click activity in the game's main execution loop and according to the turn draws the figure there.
In case of a win, we draw a line as we generally see in tic-tac-toe. For this I changed the `check_win` function a bit- Instead of just a bool indicating true or false, I take the starting point from which I will start the line and also an indicator for the type of win- row, column, diagonal or anti-diagonal. Later in the main loop, I get the returned values and I draw the lines accordingly. For drawing, inbuilt functions of pygame are used.

7.3 Connect Four

Board size 7*7. the winning criteria is to make 4 same pieces in a straight line.
same logic as tic tac toe, just the difference is here we can add new piece only at lowest available space in colomn.
So to check availibility of any column I just check whether the top row element in that column is occupied or not.
Check win logic is pretty similar to tic tac toe, just the submatrices are 4x4.
I have added an hover effect on the column by using the get mouse point function of pygame. It has a special falling piece animation which is done by running a loop in which various frames are rendered with the piece's position changing(downwards). finally it stops at its corresponding final position.

7.4 Othello

A very interesting logic is used here to know if the clicked cell is valid or not. i made a list of 8 vectors in the 8 directions(the north south east west and the in between 45 degrees)

Then at the clicked position, for each direction, i moved in that direction from the clicked position one by one in that direction checking. if its true for all directions- found a piece of same color with the previous step color being opposite color then its valid.
In this process(for a valid move), all the opposite colored pieces are at last

changed to the color of the player, by storing them in a list and then iterating on them.
The check win is called when no valid move is left, which simply counts the number of pieces of the players and returns who won.

7.5 Authentication System

The main.sh is the way to authentication in this project.The script handles login or registrations of players before launching the game. For each player, it reads a username and password, hashes the password with SHA-256, and checks it against users.tsv. New users get prompted to register with a password of at least 7 characters, returning users just get verified, and wrong passwords loop back to retry. Once both players are authenticated, their usernames are passed straight to game.py.
you can also manage your account- you can change your password or delete your user as well. Its done using if else statements and sed commands for substitution and deletion

7.6 Leaderboard

The Leaderboard.sh, Code uses awk command to read history.csv.
The awk command uses associative array to count about wins losses and win/loss ratio (equal to wins of p if losses is 0). Using formatted printing the table is printed for all games using a loop.
Another task is to print according to different metrics. For this we pass the metrics as an argument for the bash script. According to that argument an appropriate piped sort command does the sorting for the leaderboard table.
Later the Leaderboard data funcion uses the history.csv and leaderboard to make a bar graph and pie chart using MatPlotLib. To display the images of the graphs the images are saved and then are displayed using `pygame.blit()`.

8 Retrospective

8.1 axis

it was very difficult for us to understand the concept of axis, we tried a lot of things - reading documents, watched videos but still it wasnt clear visually.

Solution-i came up with a concept/theory which was working all the time. it is that for a vector of dimension=n, axis=k represent a set of the groups. (a group is the set of of those elements who have all indices identical except the kth indice.)

8.2 Othello's logic

it was too confusing that how can i make othello without using for loops , also it was difficult to check a valid move ...
Solution- it was later announced that except in check win function, we can use for loops, also i came up with an idea of the 8 direction vectors checking.

8.3 diagonal check win

since tic tac toe was 10*10, to check win the diagonals, it was so difficult to check the non-main diagonals without using for loop.
Solution-i got an idea that instead i can just check the 5*5 squares because here i wont have to check non main diagonals in 5*5, hence i made a set of all 5*5 squares and simplified it.

8.4 Text alignment with varying length of player names

it was difficult to align the text of player names with the character images in menu because the length of player names can vary and hence the text can be misaligned, which will affect the look of UI.
Solution- I defined a rectangle box for the text(using `pygame.getRect`) and aligned the text in the center of that box, hence no matter what the length of the text is, it will always be aligned in the center of the box and hence with the character image.

8.5 Arrangement according to the background image for various games

it was difficult to arrange the figures etc according to the background image for various games, because each game has different background image and hence the arrangement for one game may not look good for another game.
Solution- I made different arrangements for each game according to the background image by manually checking the offsets and side lengths, and also made some general arrangements which can be used in all games, such as the arrangement of player names and scores, etc.

9 What would we do with 10 more hours

9.1 Add new games

With 10 more hours, we could add one more board game like checkers, chess,e.t.c. there are various such game ideas available on 2 players, a mobile game

9.2 Make better UI

We could make a better UI like better sound effects, background picture, background music etc. add more animations, try to increase smoothness in animations

9.3 Optimise

We could optimise our existing code so that it could run faster, use lesser memory

9.4 Add settings

We could add settings option in menu, in which user can control various things like volume, e.t.c.

References

[1] Classes — python 3 documentation. https://docs.python.org/3/tutorial/classes.html.

[2] Connect four rules. https://boardgamegeek.com/boardgame/2719/connect-four.

[3] Connect4 rules. https://boardgamegeek.com/boardgame/2719/connect-four.

[4] Othello (reversi) rules. https://www.worldothello.org/about/about-othello/othello-rules.

[5] Othello rules. https://www.worldothello.org.

[6] Pygame ce official documentation. https://pyga.me.

[7] Sample modular project — maze. https://github.com/rennaMAhcuS/Maze.

[8] Tic-tac-toe rules. https://en.wikipedia.org/wiki/Tic-tac-toe.

10 Images

At last, here are some gameplay visuals...

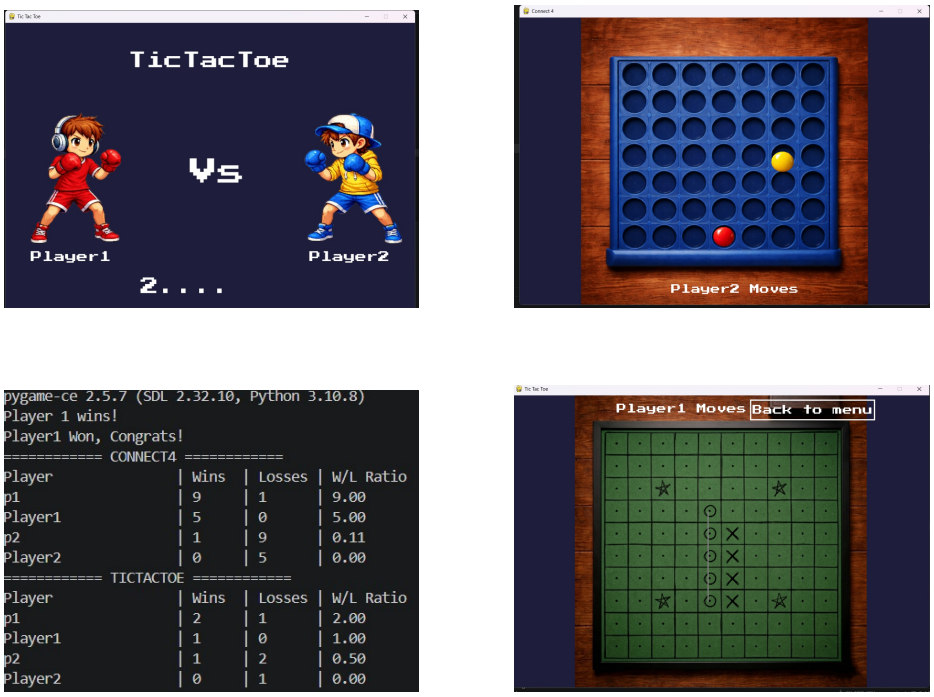


Figure 1: Gameplay visuals -1

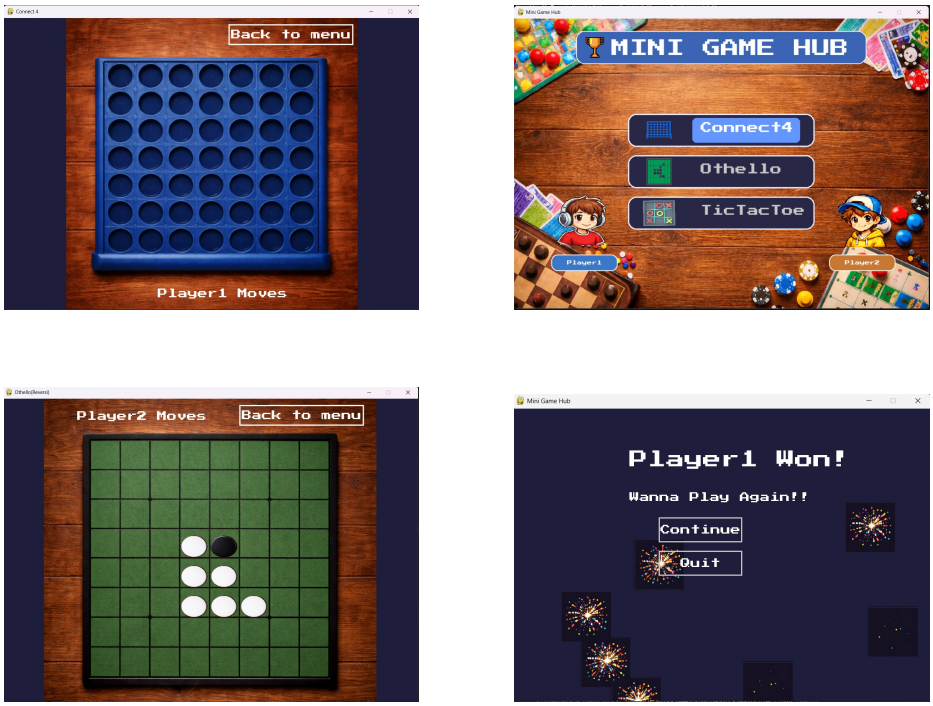


Figure 2: Gameplay visuals -2